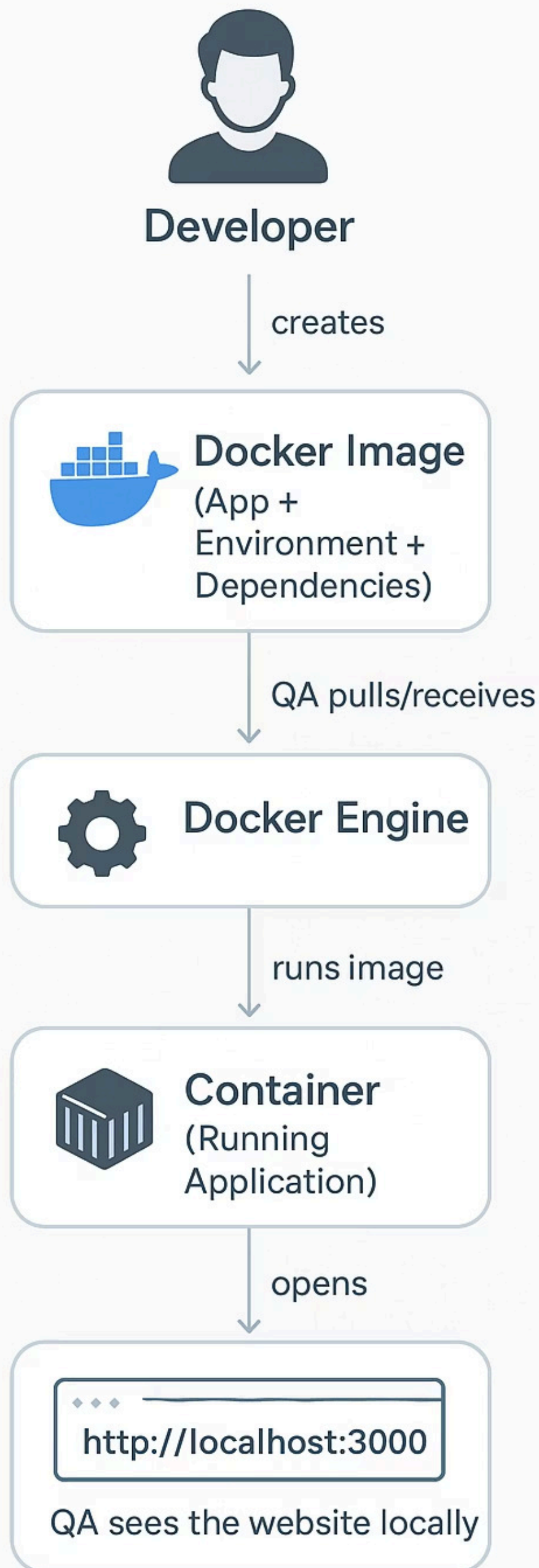


Docker Flow for Testers



Docker helps QA testers

- Image = packaged app
- Container = running app
- Localhost = QA's local test URL
- Enables early testing without staging
- Faster feedback loop for QA & Developers



How Manual Testers Use Docker to Test Websites Locally

Docker isn't just for DevOps or automation. Manual testers can also use Docker to test websites early — even before staging is ready.

Keep scrolling to learn how 

Why Docker for QA?



Traditional Testing

Normally testers wait for:

- Staging URL
- Deployment
- Functional builds



Docker Testing

With Docker you get:

- Website runs on your laptop
- Testing starts early
- See real UI before staging
- Earlier bug reports to dev team

Result: faster development + faster testing 🚀

What Developer Gives to QA

A tester does **NOT** create Docker setup. The developer provides everything you need to get started:

01

Docker Image Command

Example: `docker pull registry.company.com/app/frontend:v3`

02

Run Command

Example: `docker run -p 3000:3000`
`registry.company.com/app/frontend:v3`

03

Login Credentials

Example: admin / admin123

04

Optional Config Files

.env file or docker-compose.yml

What Tester Must Install

Only **one tool required**:

Docker Desktop

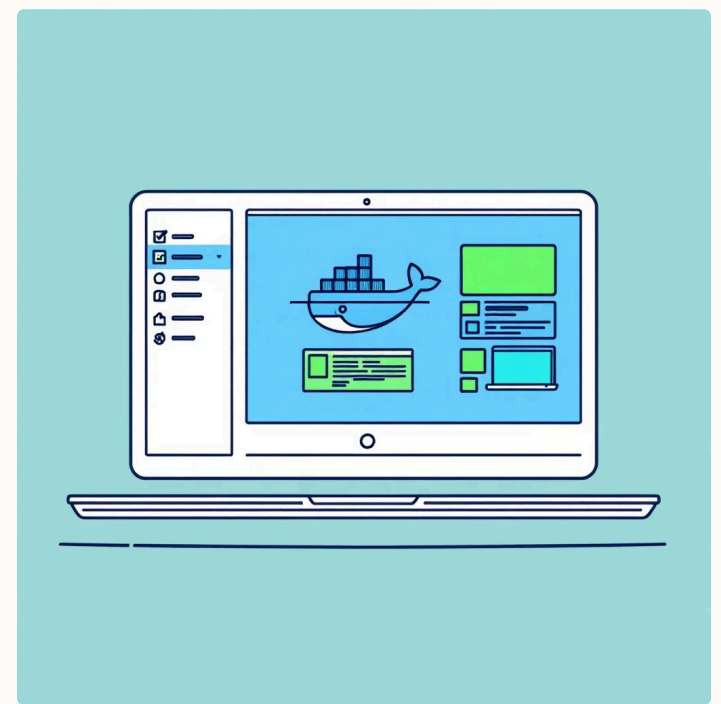
Download from:

docker.com/products/docker-desktop

System requirements:

- 8GB RAM minimum (16GB recommended)
- Windows 10/11 or Mac/Linux
- Virtualization enabled
- Internet for first-time download

No coding, no DevOps setup needed.





Step 1: Pull the Image

Open Terminal or PowerShell and run the command provided by your developer:

```
docker pull registry.company.com/app/frontend:v3
```

What Downloads:

-  Application code
-  Environment settings
-  Dependencies
-  Server setup

 Think of it like: "downloading a ready-made software box that has everything pre-configured."



Step 2: Run the Application

Use the run command given by your developer:

```
docker run -p 3000:3000 registry.company.com/app/frontend:v3
```

What Happens Now?

Docker creates a **container** — this is your running application. The container behaves like a mini-computer inside your system, completely isolated and ready for testing.



Step 3: Open Website in Browser

Once the container is running, open your browser and navigate to the local address:

Frontend

<http://localhost:3000>

Admin Panel

<http://localhost:8000>

Backend API

<http://localhost:5000/api>

🔥 This is **NOT** staging — this is **YOUR local running website**.

Step 4: Test Normally

Now test the application exactly as you would on staging or production:



UI Testing

Test interface elements and layouts



Functional Testing

Verify features work correctly



Regression

Check existing functionality



Login / Signup

Test authentication flows



Error Handling

Verify error messages



Field Validations

Check input requirements



Pro tip: You can capture logs using `docker logs <container-id>` to debug issues faster.

When Developer Updates Code

Docker images do **NOT** update automatically. When the developer pushes new changes, you need to manually refresh your local environment:

1

Stop Previous Container

```
docker stop <container-id>
```

2

Pull New Build

```
docker pull registry.company.com/app/frontend:v4
```

3

Run Again

Use the same run command with new version

💡 Always check with your developer for the **latest version number** before pulling.

Final Summary



Docker lets testers run applications locally — **without waiting for staging**



Perfect for early testing, documentation, test case creation, and **faster feedback**



Very easy to use — just **pull → run → test**

Small workflow:

Pull image → Run container → Open localhost → Test

Manual QA + Docker = Faster releases + Better product quality 💙🐳

Found this helpful? Share this with your QA team! 🔄